

Control

Programming and Music
<http://www.bjarni-gunnarsson.net>

Assignment 3

Assignment 3

The third assignment of the year contains two parts:

1. A **generative system** that can create sound material and is based on a graphical user interface and/or a midi control setup.
2. A **short etude** that is composed using the system.

The code should be handed in as well as a small text with a minimum of **300 words** explaining the approaches and the value these could have in a musical setting.

The assignment should be handed in before the the **28th of May**. (and will be discussed during this class). In the classes of **16th** of April you should already present me the direction and progress of the project. The best would be to start soon.

Assignment 3, System & GUI

The generative system must be capable of the following requirements:

- Generate *musical material* that can be used in a musical composition or live performance.
- Contain a **GUI** or use **MIDI** for its main controls.
- Create sound structures based on *reading samples* and/or *sound synthesis*.
- **Process** the generated audio with built in *effects* or sound *transformation processes*.

Assignment 3, Etude

The small composition must meet the following requirements:

- Be at least **3 minute** long.
- Use **material** created by the generative system.
- Exist as an attempt to create **something of value** to its composer (not be simply a demo or rough sketch).
- Putting the material together in another program is allowed but could also be done in SuperCollider.

Exercises

Exercises

Exercise 1: CC Filter Control

Create a SynthDef with a resonant filter processing noise. Use MIDIFunc.cc to control both the filter frequency and resonance from two different CC numbers. Use ControlSpec for the mapping.

Exercise 2: Polyphonic MIDI Keys

Create a polyphonic MIDI instrument using noteOn and noteOff. Use an array of 128 slots to track active synths. Map velocity to amplitude. Make sure notes release properly with a gate.

Exercise 3: Direction-Aware CC

Track the direction a CC knob is moving (up or down) and use that to change something about the sound — for example, switching between two filter types or panning left vs right.

Exercise 4: CC to Pattern

Use a PatternProxy inside a Pbind so that a CC knob can change a pattern parameter (pitch range, density, or duration) while the pattern is playing.

Exercises

Exercise 5: Slider Synth

Create a GUI window with a slider that controls the frequency of a filtered oscillator. Use a ControlSpec with exponential mapping and free the synth on window close.

Exercise 6: XY Pad

Create a UserView that acts as an XY pad. Mouse position controls two synth parameters (e.g. frequency and filter cutoff). Draw a crosshair or dot at the current position.

Exercise 7: Button Grid

Create a window with a grid of buttons (at least 3x3). Each button should trigger a different pitched sound when clicked. Use Pen or color to indicate the last pressed button.

Exercise 8: Sound-Reactive Display

Create a synth that sends amplitude data to a control bus. Use a UserView with `animate_(true)` to read the bus with `getSynchronous` and draw a shape whose size or color responds to the amplitude in real time.

Exercises

Exercise 5: Slider Synth

Create a GUI window with a slider that controls the frequency of a filtered oscillator. Use a ControlSpec with exponential mapping and free the synth on window close.

Exercise 6: XY Pad

Create a UserView that acts as an XY pad. Mouse position controls two synth parameters (e.g. frequency and filter cutoff). Draw a crosshair or dot at the current position.

Exercise 7: Button Grid

Create a window with a grid of buttons (at least 3x3). Each button should trigger a different pitched sound when clicked. Use Pen or color to indicate the last pressed button.

Exercise 8: Sound-Reactive Display

Create a synth that sends amplitude data to a control bus. Use a UserView with `animate_(true)` to read the bus with `getSynchronous` and draw a shape whose size or color responds to the amplitude in real time.

